LESSON 01 · GIT · 50 MIN

1교시 Git 기초 — 파일이 기록되기까지

Git 명령어를 외우는 시간이 아닙니다. 내 파일이 지금 어느 영역에 있는지 읽을 수 있게 되는 시간입니다. 이번 시간이 끝나면 커밋을 만들고, 잘못 만든 커밋을 되돌릴 수 있습니다.

개념 15분 · 실습 25분 · 정리와 질문 10분

OVERVIEW

파일이 지나가는 세 개의 방



되돌리기는 반대 방향입니다. **git restore --staged** 는 스테이징에서 작업 디렉터리로, **git restore** 는 마지막 커밋 내용으로 파일을 덮어씁니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`git init`	현재 폴더를 Git 저장소로 만든다	새 프로젝트 시작할 때 딱 한 번
`git status`	파일이 어느 영역에 있는지 보여준다	헷갈릴 때마다 · 가장 자주 쓰는 명령
`git add <파일>`	작업 디렉터리 → 스테이징	커밋에 담을 것만 골라 올릴 때
`git commit -m "메시지"`	스테이징 내용을 저장소에 기록	의미 있는 작업 단위가 끝났을 때
`git log --oneline`	커밋 기록을 한 줄씩 확인	지금까지 무엇을 했는지 볼 때
`git diff`	아직 add 안 한 변경 내용	커밋 전 무엇이 바뀌었는지 확인
`git restore <파일>`	마지막 커밋 상태로 파일 되돌리기	수정을 버리고 싶을 때 (복구 불가)
`git restore --staged <파일>`	스테이징에서만 내리기	add를 잘못했을 때 (내용은 유지)
`git reset --soft/--mixed/--hard HEAD~1`	마지막 커밋 취소 · 되돌리는 깊이가 다름	커밋을 잘못 만들었을 때

HANDS-ON

실습 순서

STEP 01 준비물 확인과 신원 설정

Git이 설치되어 있고, 내가 누구인지 Git이 알고 있는지 확인합니다.

```
git --version
git config --global user.name "본인 이름"
git config --global user.email "본인@이메일.com"
git config --global --list
```

확인 포인트

- [ ] 버전 번호가 출력된다
- [ ] 마지막 명령 결과에 user.name 과 user.email 이 보인다

이 설정을 건너뛰면 첫 커밋에서 **Author identity unknown** 오류가 납니다. 커밋에 기록될 이름이라 실명 또는 GitHub 아이디를 권장합니다.

STEP 02 저장소 만들기

빈 폴더를 Git이 감시하는 저장소로 바꿉니다.

```
mkdir git-practice
cd git-practice
git init
git status
```

확인 포인트

- [ ] Initialized empty Git repository 메시지가 나온다
- [ ] ls -a (윈도우 dir /a) 로 보면 .git 폴더가 생겼다
- [ ] status에 No commits yet 이라고 나온다

.git 폴더가 저장소의 전부입니다. 이 폴더를 지우면 모든 기록이 사라지고, 복사하면 기록까지 함께 복사됩니다.

STEP 03 파일을 만들고 상태 읽기

Git이 새 파일을 어떻게 보는지 확인합니다.

```
echo "hello git" > app.txt
git status
```

확인 포인트

- [ ] app.txt 가 Untracked files 아래에 빨간색으로 표시된다

> 빨간색 = 아직 Git이 관리하지 않거나 스테이징에 올라가지 않은 상태입니다.

STEP 04 스테이징에 올리기

add가 파일을 옮기는 것이지 저장하는 것이 아님을 확인합니다.

```
git add app.txt
git status
```

확인 포인트

- [ ] 같은 파일이 Changes to be committed 아래 초록색으로 이동했다
- [ ] 파일 내용은 하나도 바뀌지 않았다 (cat app.txt 로 확인)

STEP 05 첫 커밋 만들기

스테이징에 올라온 것만 기록으로 남깁니다.

```
git commit -m "첫 번째 커밋"
git log --oneline
git status
```

확인 포인트

- [ ] log에 7자리 해시와 메시지가 한 줄 나온다
- [ ] status가 nothing to commit, working tree clean 으로 바뀐다

STEP 06 수정하고 두 번째 커밋

add → commit 흐름을 한 번 더 몸에 익힙니다.

```
echo "두 번째 줄" >> app.txt
git status
git diff
git add app.txt
git commit -m "두 번째 줄 추가"
git log --oneline
```

확인 포인트

- [ ] git diff 가 추가된 줄을 + 로 보여준다
- [ ] add 를 한 뒤에는 git diff 결과가 비어 있다
- [ ] 커밋이 2개가 되었다

> **git diff** 는 작업 디렉터리와 스테이징의 차이만 보여줍니다. 스테이징에 올린 내용을 보려면 **git diff --staged** 를 씁니다.

STEP 07 .gitignore 로 추적에서 빼기

기록하면 안 되는 파일을 목록에서 제외합니다.

```
echo "secret.txt" > .gitignore
echo "비밀번호1234" > secret.txt
git status
git add .gitignore
git commit -m ".gitignore 추가"
```

확인 포인트

- [ ] secret.txt 가 status 목록에 나타나지 않는다
- [ ] .gitignore 자체는 새 파일로 잡힌다 (이 파일은 커밋해야 팀원과 공유됩니다)

> **이미 커밋된 파일은 .gitignore에 적어도 계속 추적됩니다.** 그때는 **git rm --cached <파일>** 로 추적을 먼저 끊어야 합니다.

STEP 08 되돌리기 두 방향 실습

restore가 어느 방향으로 움직이는지 두 경우를 비교합니다.

```
# (1) 수정을 버리기
echo "실수로 쓴 줄" >> app.txt
git restore app.txt
cat app.txt
# (2) add 를 취소하기
echo "실수로 쓴 줄" >> app.txt
git add app.txt
git restore --staged app.txt
git status
```

확인 포인트

- [ ] (1) 파일 내용이 원래대로 돌아왔다
- [ ] (2) 파일 내용은 그대로지만 초록색에서 빨간색으로 내려왔다

> **주의** · (1)의 git restore 로 버린 수정은 되살릴 수 없습니다. 커밋하지 않은 변경은 Git이 지켜 주지 않습니다.

STEP 09 reset 3형제 비교 — 오늘의 핵심

같은 커밋 취소인데 무엇이 남고 무엇이 사라지는지 직접 비교합니다.

```
# 실험용 커밋을 하나 만듭니다
echo "임시" > temp.txt && git add temp.txt && git commit -m "temp 추가"
# (1) soft — 커밋만 취소
git reset --soft HEAD~1
git status && git log --oneline
# (2) mixed — 커밋과 스테이징 취소 (기본값)
git commit -m "temp 추가" && git reset --mixed HEAD~1
git status
# (3) hard — 파일까지 되돌림
git add temp.txt && git commit -m "temp 추가" && git reset --hard HEAD~1
git status && ls
```

확인 포인트

- [ ] (1) temp.txt 가 초록색(스테이징)으로 남아 있다
- [ ] (2) temp.txt 가 빨간색(작업 디렉터리)으로 내려왔다
- [ ] (3) temp.txt 파일 자체가 사라졌다 — ls 결과에 없다

> 세 경우 모두 커밋은 1개씩 줄지만, 파일이 되돌아가는 깊이가 다릅니다. **--hard 만 파일을 잃습니다.** 실무에서 가장 조심해야 할 명령입니다.

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
`Author identity unknown`	user.name / user.email 미설정	git config --global 로 이름과 메일을 설정한 뒤 다시 커밋
`fatal: not a git repository`	git init 을 하지 않은 폴더	pwd 로 위치 확인 → 올바른 폴더로 이동하거나 git init
`nothing to commit, working tree clean`	변경이 없거나 이미 커밋됨	편집기에서 파일을 저장했는지 확인 후 git status
`error: pathspec ... did not match`	파일 이름 오타	ls 로 실제 파일명 확인 후 다시 입력
한글 파일명이 숫자로 깨져 보임	터미널 인코딩 설정	git config --global core.quotepath false

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

- ☐ git add 없이 git commit 을 하면 어떻게 되나요?
- ☐ git status 의 빨간색과 초록색은 각각 어느 영역을 뜻하나요?
- ☐ 파일 수정을 실수로 했을 때 되돌리는 명령은 무엇인가요?
- ☐ reset --soft / --mixed / --hard 중 파일이 사라지는 것은 어느 것인가요?
- ☐ .gitignore 에 적었는데도 계속 추적되는 파일이 있다면 이유는 무엇일까요?

한 줄 정리

커밋은 '저장'이 아니라 '기록의 단위'입니다. add로 담을 것을 고르고, commit으로 그 순간에 이름을 붙입니다.

다음 시간 → 2교시 — 기록이 한 줄이 아니라 여러 갈래로 나뉠 때

WORKSHEET

막힌 STEP 번호, 화면에 나온 오류 메시지, 강사에게 물어볼 것을 적어 두세요.